Lab Eight – Probability

```
library(tidyverse)
library(patchwork)
library("extraDistr")
```

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 The Binomial and Negative Binomial Distributions

1.1 The Binomial Distribution

In many introductory courses, we make the simplifying assumption of independence in Bernoulli trials. For example, when playing best two-out-of-three legs in a darts match, we would assume Player A has an 70% chance of winning each leg. We write:

$$X \sim \text{Binomial}(n = 3, p = 0.70)$$

where

- X is the number of legs won
- n is the number of trials
- p is the probability of the event of interest

Of note, it is not necessarily the case that the players play all three legs – if either player wins legs one and two the match ends.

There are eight possible outcomes for Player A:

- (win, win, "win") $\longrightarrow$ Player A wins
- (win, win, "lose") $\longrightarrow$ Player A wins
- (win, lose, win) $\longrightarrow$ Player A wins
- (win, lose, lose) $\longrightarrow$ Player A loses
- (lose, win, win) $\longrightarrow$ Player A wins
- (lose, win, lose) $\longrightarrow$ Player A loses
- (lose, lose, "win") $\longrightarrow$ Player A loses
- (lose, lose, "lose") $\longrightarrow$ Player A loses

Note: I use quotations above to denote outcomes where the last match isn't actually played.

Use the Binomial distribution to compute the probability that Player A wins the match. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`.

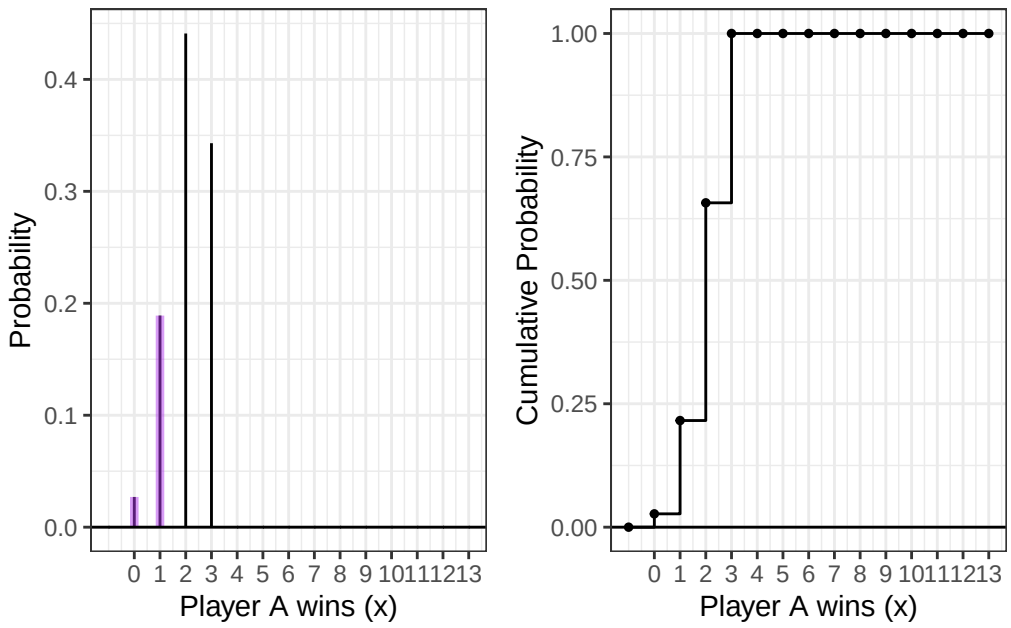
Solution

We use the binomial distribution with $n = 3, p = 0.70$ to find the probability of Player A winning ≥ 2 games:

$$P(X \geq 2)$$

We can use the PMF for this because it is a discrete variable, which allows us to find $P(X = 0)$ and $P(X = 1)$, which we can add together to get $P(X \geq 2) = 1 - P(0) - P(1)$. Now that we know how we want to find the answer, let's plot and compute it:

```r
n <- 3
p <- 0.70
ggdat <- tibble(x = -1:13) |>
mutate(PMF = dbinom(x, size=n, prob=p),
CDF = pbinom(x, size=n, prob=p))

PMF.plot <- ggplot() +
geom_linerange(data = ggdat,
aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==0|x==1),
aes(x = x, ymin = 0, ymax = PMF),
color="darkorchid2", linewidth=1.5,alpha=0.5) +
geom_hline(yintercept = 0) +
scale_x_continuous("Player A wins (x)",
breaks = seq(0, 13, 1)) +
ylab("Probability") +
theme_bw()

CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
scale_x_continuous("Player A wins (x)",
breaks = seq(0, 13, 1)) +
ylab("Cumulative Probability") +
theme_bw()
PMF.plot | CDF.plot
```



```r
# 1-PMF(0)-PMF(1)

(A_win = 1-dbinom(x = 0, prob = 0.7, size = 3)-dbinom(x = 1, prob = 0.70, size = 3) )
```

```
[1] 0.784
```

The probability of Player A winning is 0.784.

1.2 The Negative Binomial Distribution

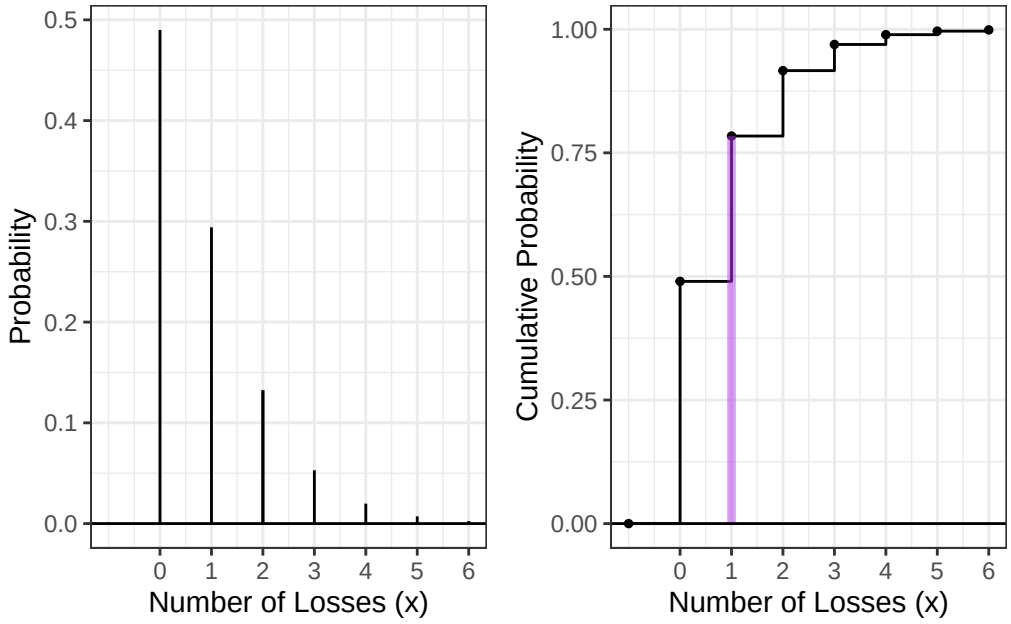
In intermediate courses, we learn the Negative Binomial distribution that we use to model the number of "failures" until the rth success. Note that this approach still makes the simplifying assumption of independence in Bernoulli trials.

Specify the two cases necessary to compute the probability. Fully explain these probability statements in the context of a best two-out-of-three match and the Negative Binomial distribution. Finally, compute the probability that Player A wins the match and show you get the same result as the Binomial approach. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`.

Solution

This situation is asking for the number of failures until the second success, but we must limit our amount of trials to 3. The parameters for the negative binomial distribution are p and r, in this case we have $p = 0.7$ and $r = 2$. We are computing the probability that it takes less than or equal to 3 trials for 2 successes to occur. This time, let's use the CDF. We want $P(X \leq 3)$, which corresponds to the CDF evaluated at 1. Let's graph and compute it:

```r
r <- 2
p <- 0.70
ggdat <- tibble(x = -1:6) |>
mutate(PMF = dnbinom(x, size=r, prob=p),
CDF = pnbinom(x, size=r, prob=p))

PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_hline(yintercept = 0) +
scale_x_continuous("Number of Losses (x)",
breaks = seq(0, 6, 1)) +
ylab("Probability") +
theme_bw()

CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_linerange(data = ggdat |> filter(x==1),
aes(x = x, ymin = 0, ymax = CDF),
color="darkorchid2", linewidth=1.5,alpha=0.5) +
geom_hline(yintercept = 0) +
scale_x_continuous("Number of Losses (x)",
breaks = seq(0, 6, 1)) +
ylab("Cumulative Probability") +
theme_bw()

PMF.plot | CDF.plot
```




```
# CDF Evaluated at X=1, giving us the probability of 1 failure:

pnbinom(q = 1, prob = 0.70, size = 2)
```

```
[1] 0.784
```

We get the same probability of 0.784. Note that the plot goes to 6 trials after which Player A is extremely likely to have won at least 2 games. We only play 3 times, but that is not relevant to the shape of the distribution. Rather, that information is encoded in the fact that we are looking for the probability of 2 successes with at most one failure.

2 The Beta Binomial Distribution

One approach to generalize this modeling is to use a Beta-Binomial distribution. This distribution arrises when we let p be a random variable that follows a Beta distribution:

$$P \sim \text{Beta}(\alpha, \beta)$$
$$X \sim \text{Binomial}(n, P).$$

This distribution is not part of base `R`, but is available using the `_bbinom` functions from the `extraDistr` package (Wolodzko 2026).

The primary benefit is that the Beta-Binomial enables us to specify some uncertainty about the skill of Player A, as p does not have to be fixed (e.g., $p = 0.70$). Suppose the result of the best two-out-of-three match follows a Beta-Binomial distribution where $\alpha = 7$ and $\beta = 3$.

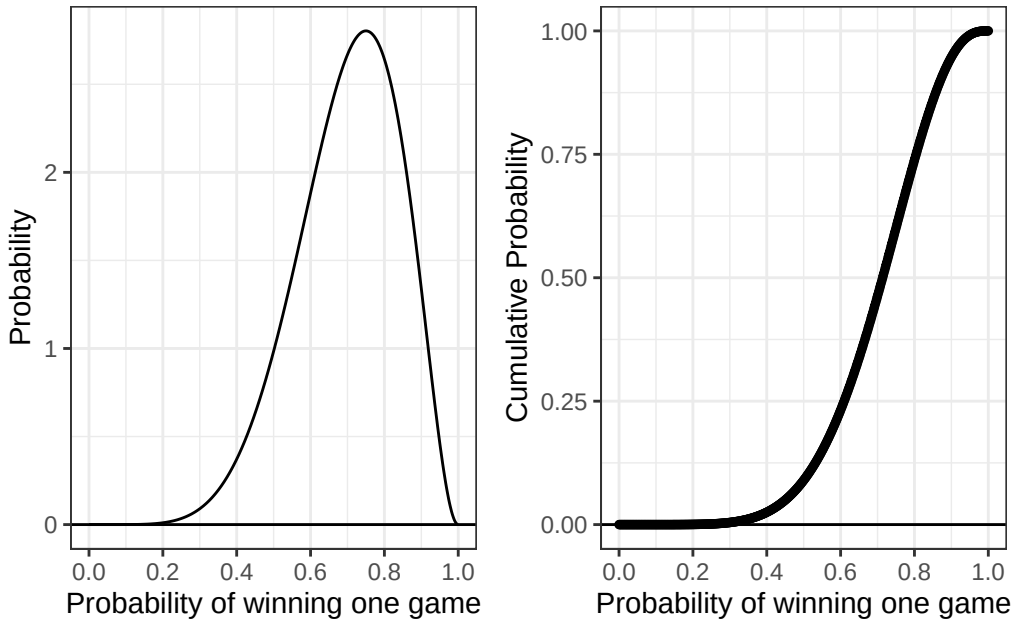
2.1 The Beta Part

1. **Plot the corresponding Beta distribution. Interpret the plot in the context of a best two-out-of-three match and the Binomial distribution.**
   Hint: The Beta distribution is available in base `R` via `_beta()`.

Solution

```
alpha = 7
beta = 3
ggdat <- tibble(x = seq(0,1, length.out=500)) |>
mutate(PDF = dbeta(x = x, shape1 = alpha, shape2 = beta),
```

```r
CDF = pbeta(q = x, shape1 = alpha, shape2 = beta))

PDF.plot <- ggplot(ggdat) +
geom_line(aes(x = x, y = PDF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PDF)) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning one game",
breaks = seq(0, 1, 0.2)) +
ylab("Probability") +
theme_bw()

CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning one game",
breaks = seq(0, 1, 0.2)) +
ylab("Cumulative Probability") +
theme_bw()

PDF.plot | CDF.plot
```



The plot on the left represents the density of the probability that Player A will win one game. For example, there is a peak around .7, which means that it is most likely that Player A will have a 70% chance to win one game. The plot on the right represents the probability that Player A will have a probability less than or equal to the x-value for winning one game. For exampe, we interpret the CDF of 1 being 1 because there is a 100% probability that Player A will have a probability of less than or equal to 1 for winning one game.

2. How does the Beta incorporate uncertainty about the skill of Player A? **Compute the mean and variance of the distribution.**

Hint: You will find the `integrate(...)` function helpful here.

$$E(P) = \mu_P = \int_0^1 p f_P(p) dp$$

$$var(P) = \sigma_P^2 = \int_0^1 (p - \mu_P)^2 f_P(p) dp$$

Solution

```
alpha = 7
beta = 3

# Mean:

(E = (integrate(function(p){
  p * dbeta(p, shape1 = alpha, shape2 = beta)
},
0,1
))$value)
```

```
[1] 0.7
```

```
#Variance:

(var = integrate(function(p){
  ((p-E)^2)*dbeta(p,alpha,beta)
},
0,1))$value
```

```
[1] 0.01909091
```

$$E(P) = \mu_P = 0.7$$

and

$$var(P) = \sigma_P^2 = 0.01909091$$

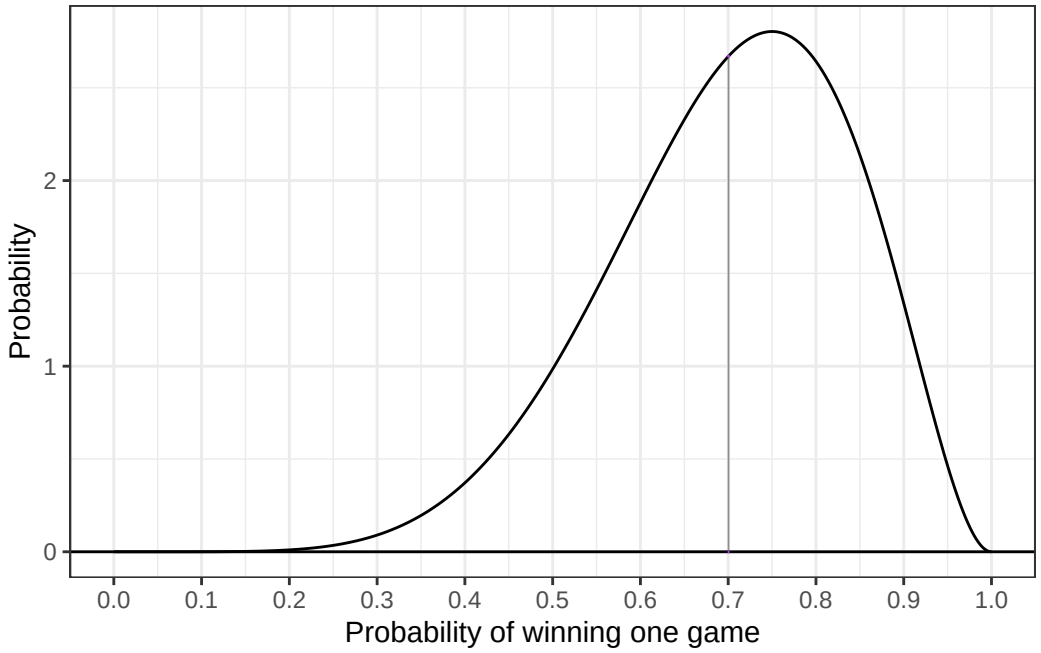
3. **What is the probability that Player A has a win probability of exactly 0.70? Make sure write out your probability statement and work to write it in terms of the distribution function (PDF/CDF) to plot the probability using `ggplot`.**
   Hint: Think carefully about the properties of continuous distributions here.

Solution

The probability that Player A has a win probability of *exactly* 0.7 is zero. The probability that a continuous distribution is equal to any value is always zero. In other words, the $P(p = 0.7)$ will be equal to zero. Let's show this with a plot:

```
alpha = 7
beta = 3
ggdat <- tibble(x = seq(0,1, length.out=500)) |>
mutate(PDF = dbeta(x = x, shape1 = alpha, shape2 = beta),
CDF = pbeta(q = x, shape1 = alpha, shape2 = beta))

PDF.plot <- ggplot(ggdat) +
geom_line(aes(x = x, y = PDF)) +
geom_hline(yintercept = 0) +
geom_ribbon(data = ggdat |> filter(abs(x-0.7)<0.002),
aes(x = x, ymin = 0, ymax = PDF),
color="darkorchid2", alpha=0.5) +
```

```
scale_x_continuous("Probability of winning one game",
breaks = seq(0, 1, 0.1)) +
ylab("Probability") +
theme_bw()

PDF.plot
```



The width of the bar is zero, i.e. we are integrating with our lower bound equal to our upper bound, so our result will be zero.

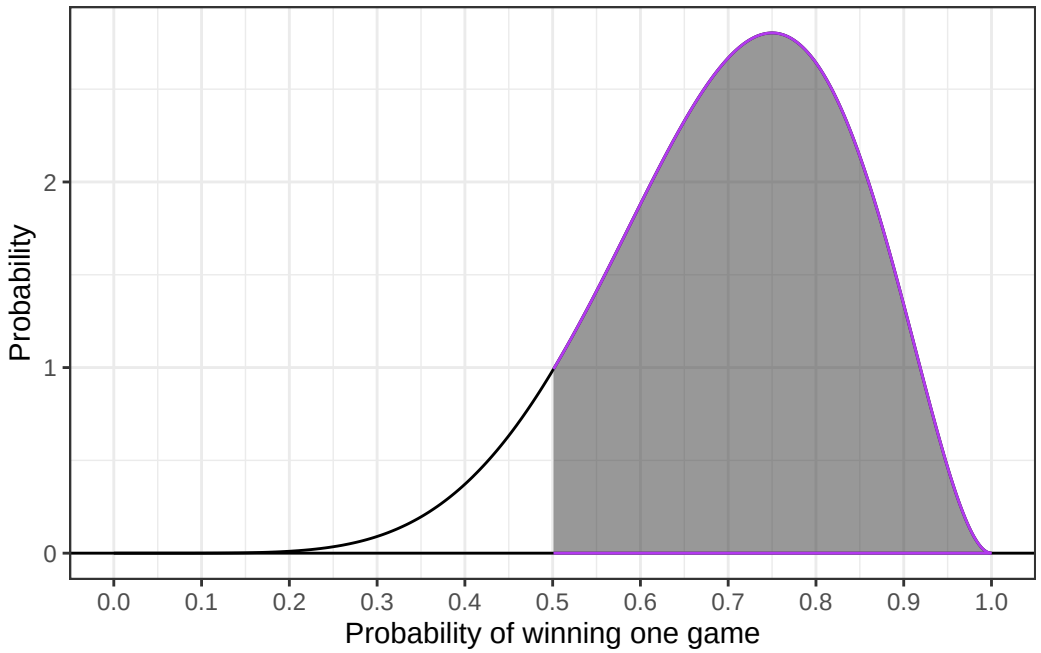
4. **What is the probability that Player A has a win probability of at least 0.50? Make sure write out your probability statement and work to write it in terms of the distribution function (PDF/CDF) to plot the probability using `ggplot`.**

Solution

The probabulity that Player A has a win probability of at least 0.5 is equal to the $1 - P(X \leq 0.5)$, and $P(X \leq 0.5)$ is equal to the CDF evaluated at 0.5. Note that the equals is trivial because the distribution is continuous. This is equivalent to the area under the PDF from 0.5 to 1, which is easier to visualize. Let's compute and plot it:

```
alpha = 7
beta = 3
ggdat <- tibble(x = seq(0,1, length.out=500)) |>
mutate(PDF = dbeta(x = x, shape1 = alpha, shape2 = beta),
CDF = pbeta(q = x, shape1 = alpha, shape2 = beta))

PDF.plot <- ggplot(ggdat) +
geom_line(aes(x = x, y = PDF)) +
geom_hline(yintercept = 0) +
geom_ribbon(data = ggdat |> filter(x>0.5),
aes(x = x, ymin = 0, ymax = PDF),
color="darkorchid2", alpha = 0.5) +
scale_x_continuous("Probability of winning one game",
breaks = seq(0, 1, 0.1)) +
ylab("Probability") +
theme_bw()

```

```
PDF.plot
```




```
(prob = 1-pbeta(q = 0.5, shape1 = alpha, shape2 = beta))
```

```
[1] 0.9101562
```

So the probability that Player A has at least a probability of 0.5 is 0.9101562.

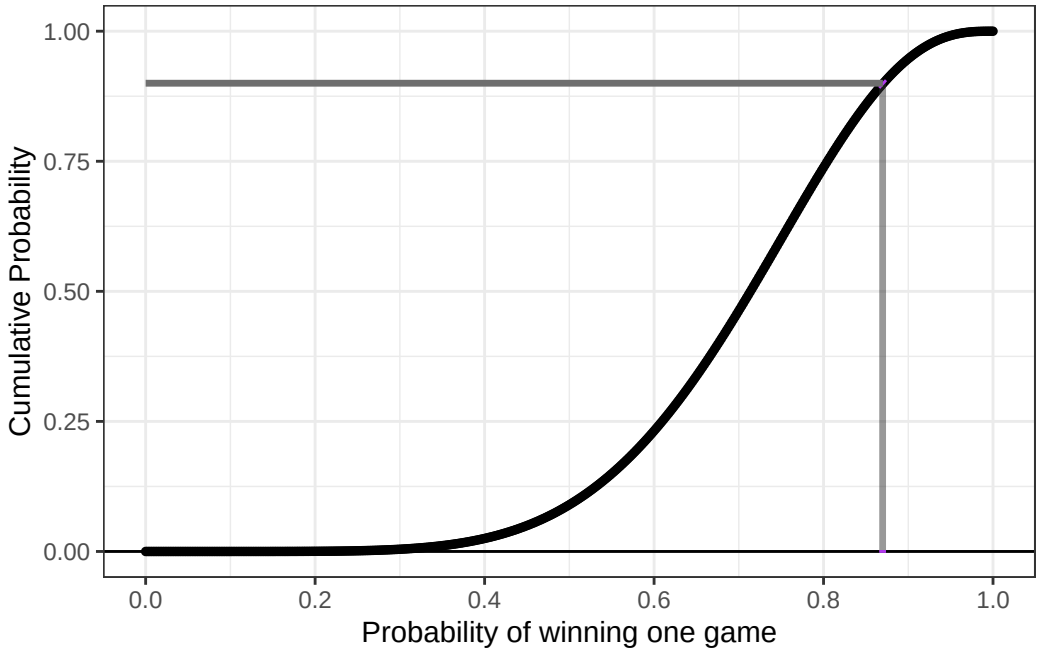
5. **What is the 90th percentile of Player A's win probability based on this Beta distribution? Make sure write out your probability statement and work to write it in terms of the distribution function (PDF/CDF) to plot the percentile using `ggplot`.**

Solution

We can use the inverse CDF to find the 90th percentile of Player A's win probability. This requires us to find where the CDF is equal to 0.9. We use the `qbeta` function:

```
alpha = 7
beta = 3
ggdat <- tibble(x = seq(0,1, length.out=500)) |>
mutate(PDF = dbeta(x = x, shape1 = alpha, shape2 = beta),
CDF = pbeta(q = x, shape1 = alpha, shape2 = beta))

CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
geom_ribbon(data = ggdat |> filter(abs(x-0.87)<0.005),
aes(x = x, ymin = 0, ymax = CDF),
color="darkorchid2", alpha = 0.5) +
geom_segment(data = ggdat |> filter(abs(x - 0.87) < 0.001),
  aes(x = 0, xend = 0.87, y = 0.9, yend = 0.9),
  color = "gray43",linewidth = 1.2) +
scale_x_continuous("Probability of winning one game",
breaks = seq(0, 1, 0.2)) +
ylab("Cumulative Probability") +
theme_bw()

```

```
CDF.plot
```




```
(inv = qbeta(p = 0.9, alpha, beta))
```

```
[1] 0.8705027
```

We can see that the 90th percentile of Player A's win probability is 0.8705027.

2.2 The Beta-Binomial Part

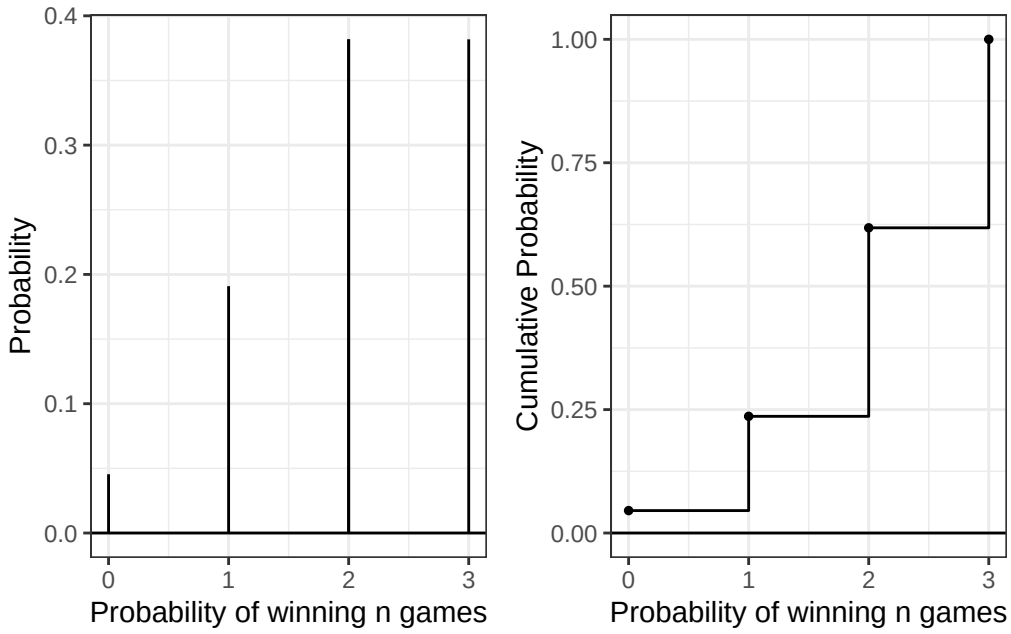
Compute the probability that Player A wins the match. Compare your result to the Binomial and Negative Binomial results. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`

Solution

We use the same logic that we used earlier for the binomial distribution. Using the Beta-Binomial distribution allows us to account for variance in p, but we do not need to change any of the probability statement logic because it is still discrete. Let's compute and plot the results:

```
alpha = 7
beta = 3
ggdat <- tibble(x = (0:3)) |>
mutate(PMF = dbbinom(x = x, size = 3, alpha=alpha, beta=beta),
CDF = pbbinom(q = x, size = 3, alpha=alpha, beta=beta))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 6, 1)) +
ylab("Probability") +
theme_bw()

```

```r
CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 6, 1)) +
ylab("Cumulative Probability") +
theme_bw()

PMF.plot | CDF.plot
```




```r
#1-PMF(0)-PMF(1)

(A_win = 1-dbbinom(x = 0, size = 3, alpha, beta)-dbbinom(x = 1, size = 3, alpha, beta))
```

```
[1] 0.7636364
```

We obtain a probability of 0.763636... ≈ 0.764 that player A wins the match. This is slightly lower than the binomial and the negative binomial result of 0.784, but similar. This makes sense, as the beta distribution is left-skewed.

2.3 The Why

Check the expected value and variance of each distribution.

Analytically, we have

$$E(X) = np \qquad var(X) = np(1-p) \qquad \text{(Binomial Distribution)}$$

$$E(X) = n\left(\frac{\alpha}{\alpha+\beta}\right) \qquad var(X) = \frac{n\alpha\beta(\alpha+\beta+n)}{(\alpha+\beta)^2(\alpha+\beta+1)} \qquad \text{(Beta-Binomial Distribution)}$$

For each distribution, compute the mean number of legs won for player A.

Hint: You can perform the sum over the finite support set here.

$$E(X) = \mu_X = \sum_{i=0}^{3} i f_X(i)$$

$$var(X) = \sigma_X^2 = \sum_{i=0}^{3} (i - \mu_X)^2 f_X(i)$$

Solution

```
#Check expected value of Binomial Distribution


(mu.binom = 0*dbinom(x= 0, size= 3, prob= 0.7) + 1*dbinom(x= 1, size= 3, prob= 0.7) + 2*dbinom(x= 2, size=
```

```
[1] 2.1
```

```
(np = 0.7 * 3)
```

```
[1] 2.1
```

```
# We get equality!

# Now, check variance of Binomial Distribution

(var.binom = (0-mu.binom)^2*dbinom(x= 0, size= 3, prob= 0.7)+ (1-mu.binom)^2*dbinom(x= 1, size= 3, prob= 0.
```

```
[1] 0.63
```

```
(analytic.var.binom= 3*0.7*(1-0.7))
```

```
[1] 0.63
```

```
# We get equality!

# Now we check the expected value of the Beta-Binomial Distribution.


(mu.bb = 0*dbbinom(x = 0, size = 3, alpha, beta) + 1*dbbinom(x = 1, size = 3, alpha, beta) + 2*dbbinom(x =
```

```
[1] 2.1
```

```
n = 3
a = 7
b = 3

(mu.bb.analytic = n*(a/(a+b)))
```

```
[1] 2.1
```

```
# We get equality! Note that this is the same as the Binomial Distribution.

# Now we check the variance of the Beta-Binomial Distribution.

(var.bb = (0-mu.bb)^2*dbbinom(x = 0, size = 3, alpha, beta) + (1-mu.bb)^2*dbbinom(x = 1, size = 3, alpha, b
```

```
[1] 0.7445455
```

```
(var.bb.analytic = (n*a*b*(a+b+n))/((a+b)^2*(a+b+1)))
```

```
[1] 0.7445455
```

```
# We get equality! Notice that this is larger than the variance of the Binomial Distribution, which makes s
```

3 Altham's Multiplicative Binomial Distribution

While the Beta-Binomial enables us to model cases where winning has a clustering effect (e.g., a "hot hand").

3.1 Probability Mass Function

The mass function for this distribution is below. While it used to be available in the `MBD` package for `R`, this package is no longer available.

$$f_X(x) = \frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)} I(x \in \{0, 1, ..., n\})$$

where

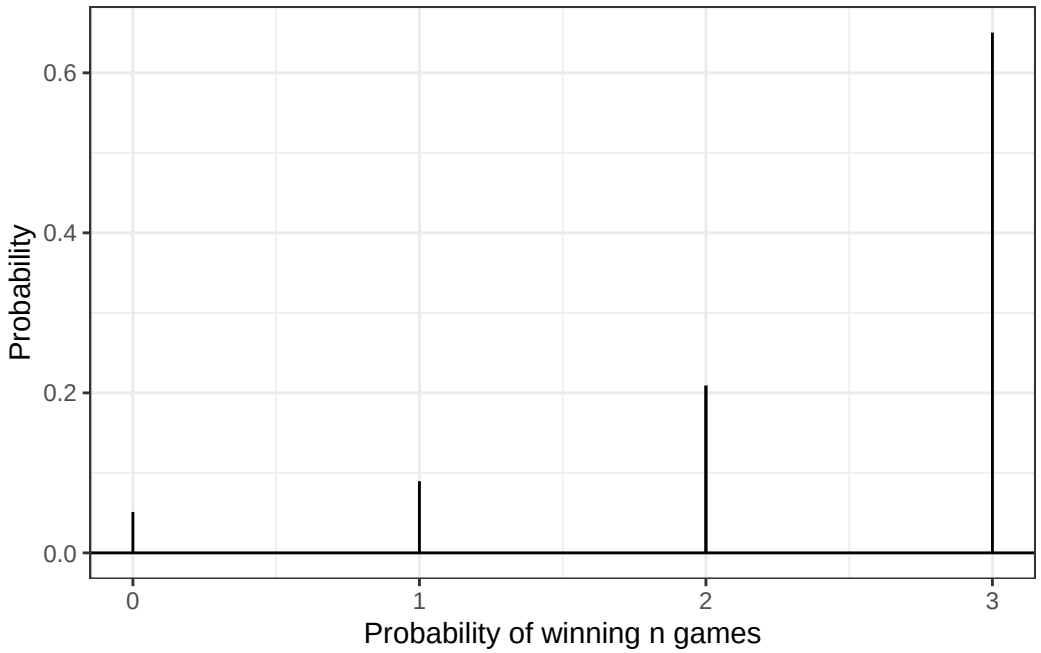
$$c = \sum_{i=0}^{n} \binom{n}{i} p^i (1-p)^{n-i} \theta^{i(n-i)}$$

Write a function in `R` called `daltbinom()` for computing the PMF of this distribution. Use your function to plot the distribution where $p = 0.70$ where $\theta = 0.50$, $\theta = 1$, and $\theta = 1.50$. Comment on the differences in the context of a best two-out-of-three match

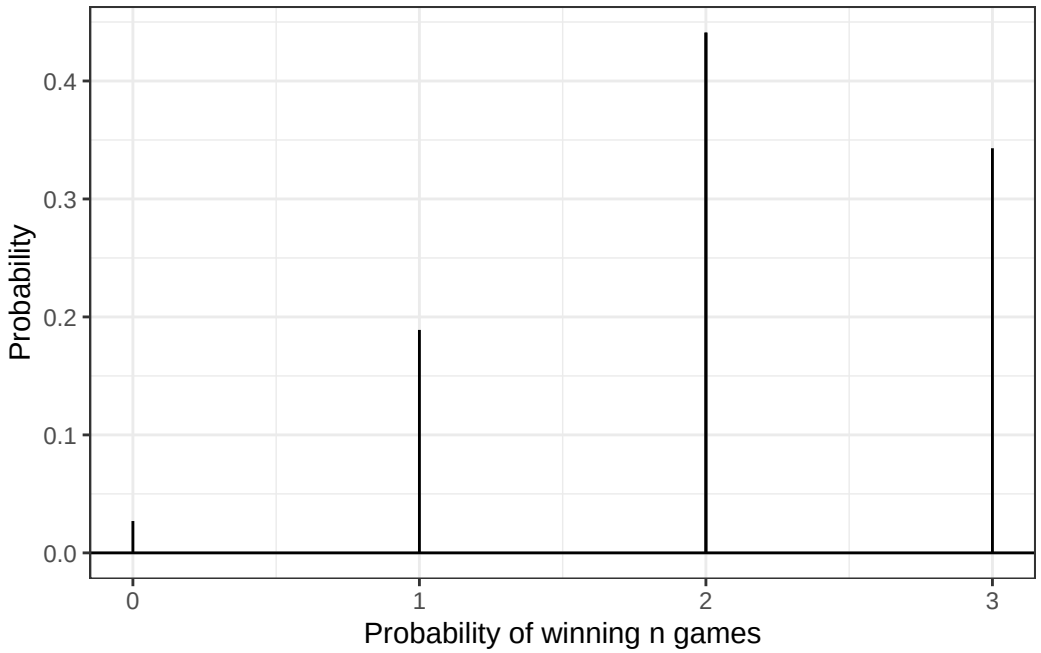
Solution

```
daltbinom= function(p,theta,n,x){
  summands = 0:n
  cvec = c(0)
  for (i in summands){
    cvec[i+1] = choose(n,i)*p^i*(1-p)^(n-i) * theta^(i*(n-i))
    }
  c=sum(cvec)
  PMF = (1/c)*choose(n,x)*p^x*(1-p)^(n-x)*theta^(x*(n-x))
return(PMF)
}
```

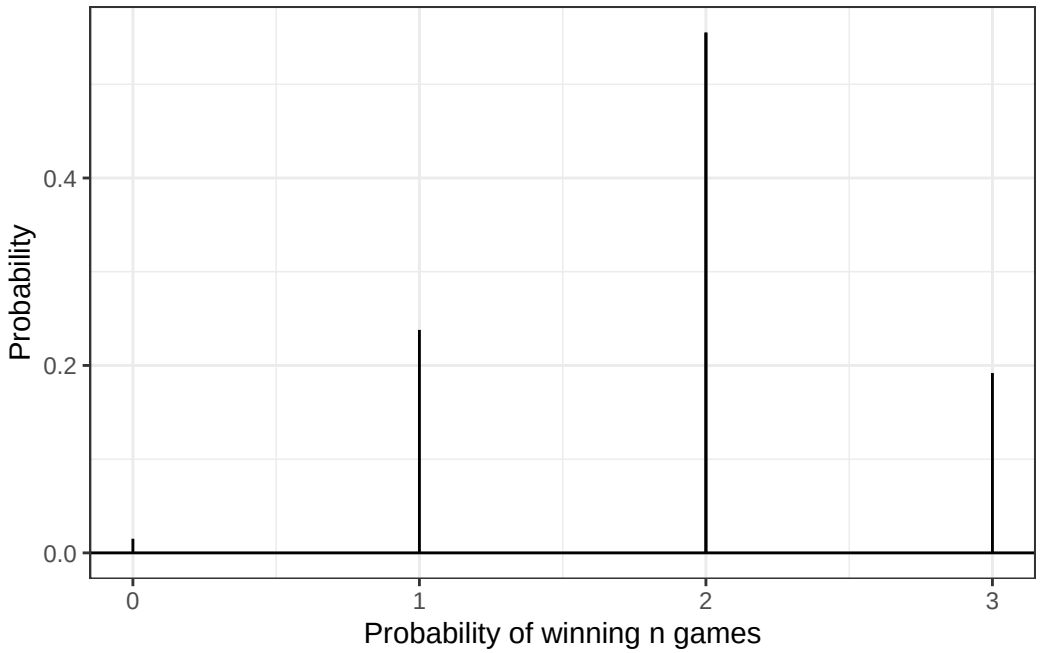
```
ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(PMF = daltbinom(x = x, p = 0.7, theta = 0.5, n=3))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Probability") +
theme_bw()

PMF.plot
```




```r
ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(PMF = daltbinom(x = x, p = 0.7, theta = 1, n=3))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Probability") +
theme_bw()

PMF.plot
```




```r
ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(PMF = daltbinom(x = x, p = 0.7, theta = 1.5, n=3))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Probability") +
theme_bw()

PMF.plot
```



A higher theta seems to correspond to a shorter or less potent hot streak, or maybe there is some observed "cost" to having a hot streak. We will be able to see better in the next section if probability of winning increases with theta for these values.

3.2 Cumulative Distribution Function

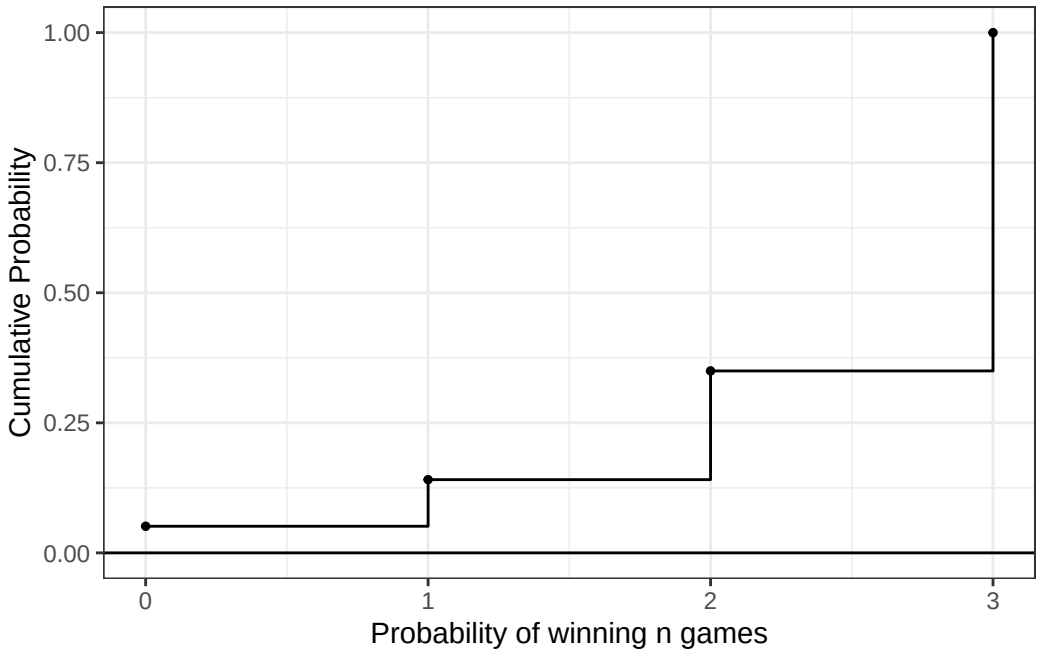
Use your `daltbinom()` function to create a `paltbinom()` function for computing the CDF of this distribution. Use your function to plot the distribution where $p = 0.70$ where $\theta = 0.50$, $\theta = 1$, and $\theta = 1.50$.

Solution

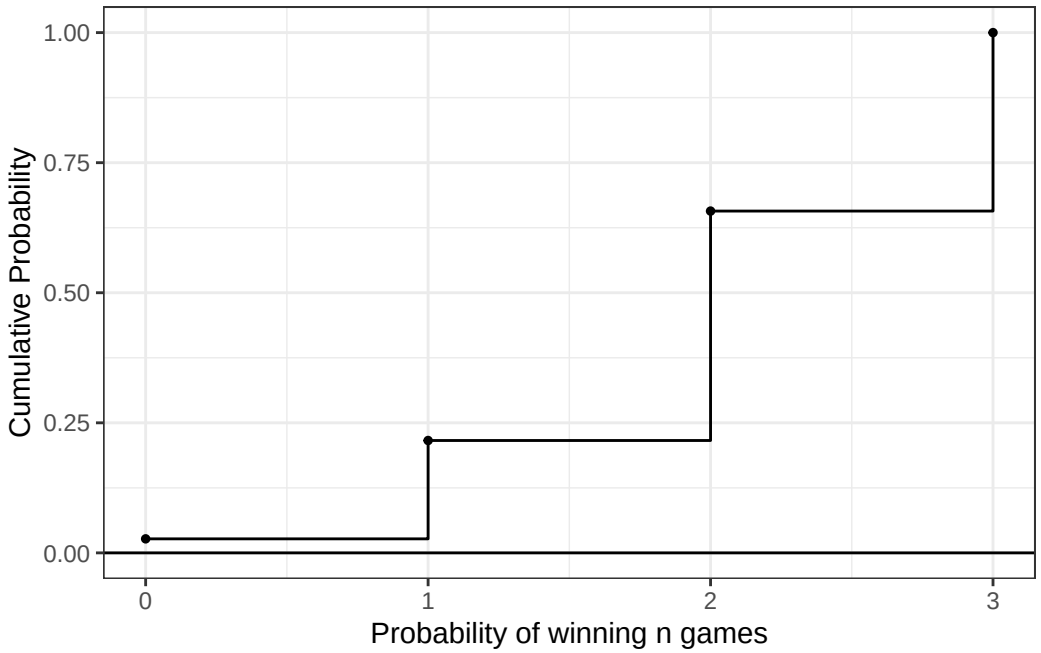
```
paltbinom = function(p,theta,n,x){
  seq = 0:x
  CDF = sum(daltbinom(p,theta,n,x=seq))
  return(CDF)
}
```

```
ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(CDF =sapply(x, function(k) paltbinom(p=0.7, theta=0.5, n=3, k)))
```

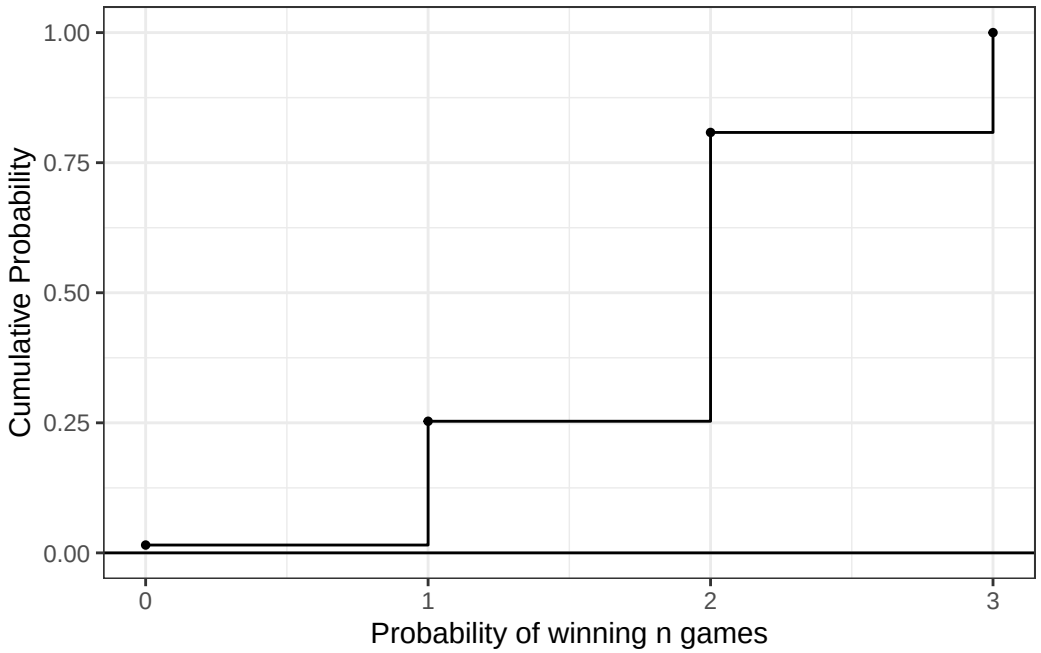
```
CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Cumulative Probability") +
theme_bw()

CDF.plot
```




```r
ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(CDF =sapply(x, function(k) paltbinom(p=0.7, theta=1, n=3, k)))



CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Cumulative Probability") +
theme_bw()

CDF.plot
```




```r
ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(CDF =sapply(x, function(k) paltbinom(p=0.7, theta=1.5, n=3, k)))



CDF.plot <- ggplot(ggdat) +
geom_step(aes(x = x, y = CDF)) +
geom_point(aes(x = x, y = CDF), size = 1) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Cumulative Probability") +
theme_bw()

CDF.plot
```



It appears that a higher theta lowers the probability of Player A winning 0 games, but raises the probability of Player A winning one game. We will see in the next section whether theta has a positive or negative effect on the probability of Player A winning in a best 2-out-of-3 scenario.

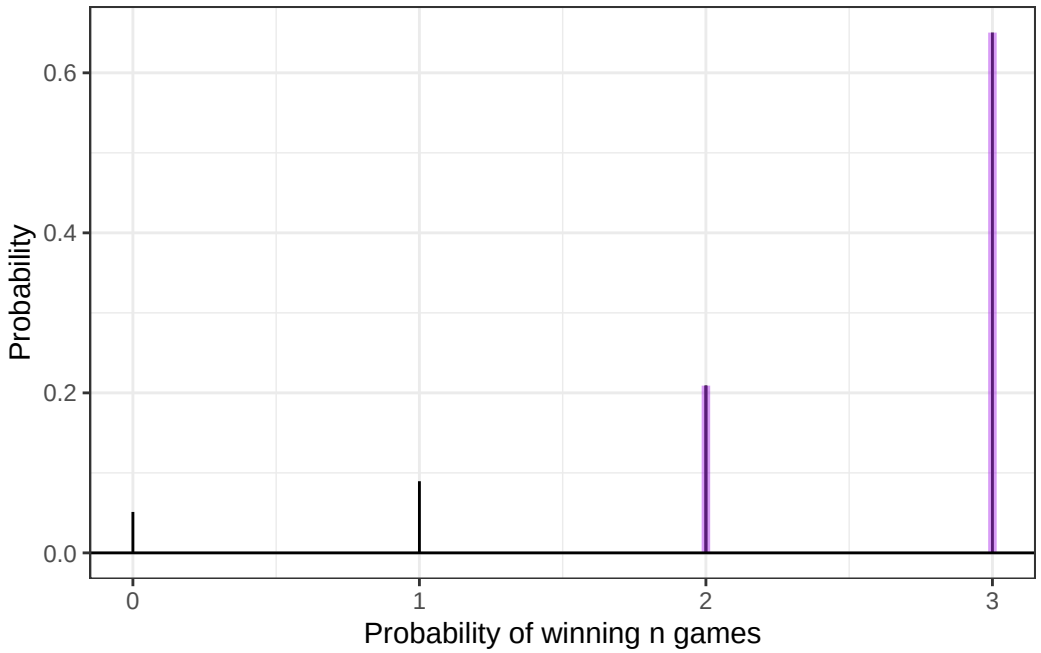
3.3 The Winning Probability

Compute the probability that Player A wins the match using Altham's Multiplicative Binomial Distribution. Compare your result to the Binomial and Negative Binomial results. Compare your results to the Beta-Binomial model, too. Make sure write out your probability statement and work to write it in terms of the distribution function (PMF/CDF) to plot the probability using `ggplot`.

Solution

To solve this, we can use the PMF to find the probability of winning 0 or 1 games and subtract the probability of these outcomes from 1, i.e. $1 - F_X(1) - F_X(0)$

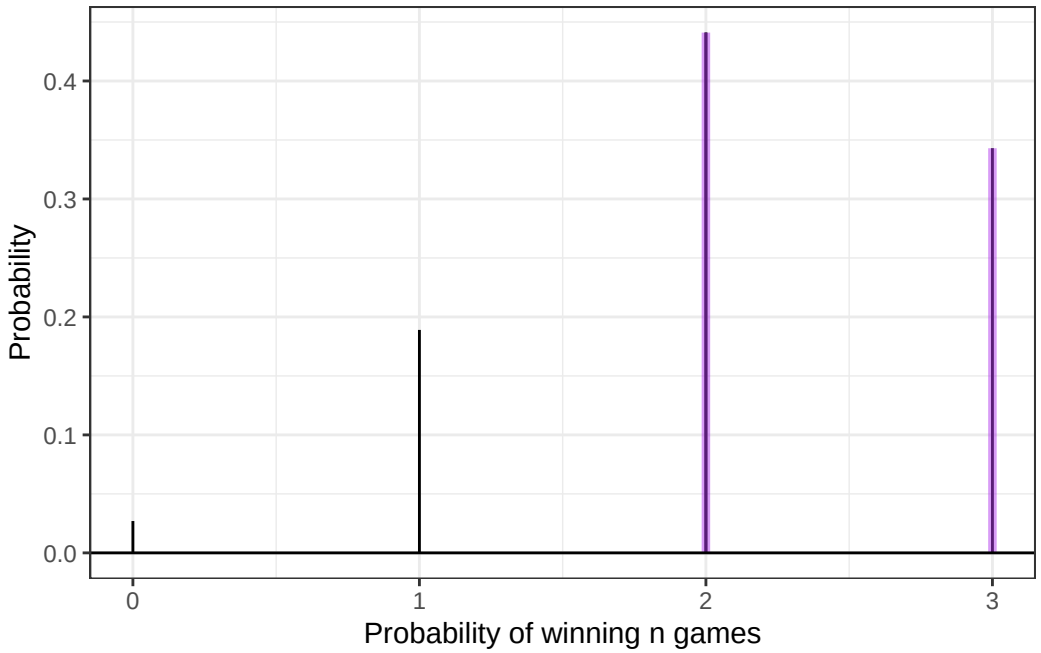
```
# for theta = 0.5:

ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(PMF = daltbinom(x = x, p = 0.7, theta = 0.5, n=3))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2|x==3),
aes(x = x, ymin = 0, ymax = PMF),
color="darkorchid2", linewidth=1.5,alpha=0.5) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Probability") +
theme_bw()

PMF.plot
```




```
(A_win = 1 - daltbinom(p=0.7, theta=0.5, n=3, x=0) -
           daltbinom(p=0.7, theta=0.5, n=3, x=1))
```

```
[1] 0.8592417
```

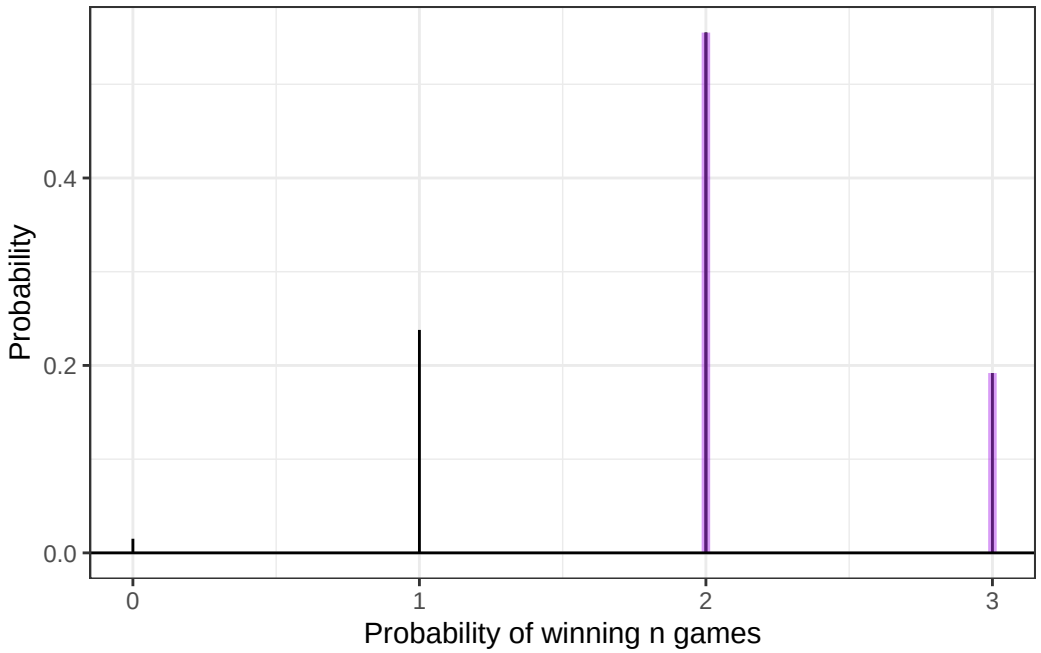
```
# for theta = 1:

ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(PMF = daltbinom(x = x, p = 0.7, theta = 1, n=3))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2|x==3),
aes(x = x, ymin = 0, ymax = PMF),
color="darkorchid2", linewidth=1.5,alpha=0.5) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Probability") +
theme_bw()

PMF.plot
```




```
(A_win = 1 - daltbinom(p=0.7, theta=1, n=3, x=0) -
           daltbinom(p=0.7, theta=1, n=3, x=1))
```

```
[1] 0.784
```

```
# for theta = 0.5:

ggdat <- tibble(x = c(0,1,2,3)) |>
mutate(PMF = daltbinom(x = x, p = 0.7, theta = 1.5, n=3))


PMF.plot <- ggplot(ggdat) +
geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2),
aes(x = x, ymin = 0, ymax = PMF)) +
geom_linerange(data = ggdat |> filter(x==2|x==3),
aes(x = x, ymin = 0, ymax = PMF),
color="darkorchid2", linewidth=1.5,alpha=0.5) +
geom_hline(yintercept = 0) +
scale_x_continuous("Probability of winning n games",
breaks = seq(0, 3, 1)) +
ylab("Probability") +
theme_bw()

PMF.plot
```




```r
(A_win = 1 - daltbinom(p=0.7, theta=1.5, n=3, x=0) -
           daltbinom(p=0.7, theta=1.5, n=3, x=1))
```

```
[1] 0.746993
```

We get win (Player A wins in a best two-out-of-three scenario) probabilities of $\approx 0.86, 0.78$, and 0.75 for $\theta = 0.5, 1$, and 1.5, respectively. Our Beta-Binomial result was 0.764, our Negative Binomial result was 0.784, and this was the same as our Binomial result. The Altham distribution is highly dependent on theta, and we get some results with a much greater probability of winning ($\theta = 0.5$), and some results with about the same probability.

4 Altham's Multiplicative Beta-Binomial Distribution

You may have wondered – "Can we do the Beta thing with Altham's version of the Binomial Distribution?" The answer is yes!

4.1 Probability Mass Function

The mass function for this distribution is below.

$$f_X(x) = \left[\int_0^1 dp \left(\frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)}\right) \left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1}\right)\right] I(x \in \{0, 1, ..., n\})$$

Write a function in `R` called `daltbbinom()` for computing the PMF of this distribution. Use your function to plot the distribution where $\alpha = 7$ and $\beta = 3$ where $\theta = 0.50$, $\theta = 1$, and $\theta = 1.50$. Comment on the differences in the context of a best two-out-of-three match.

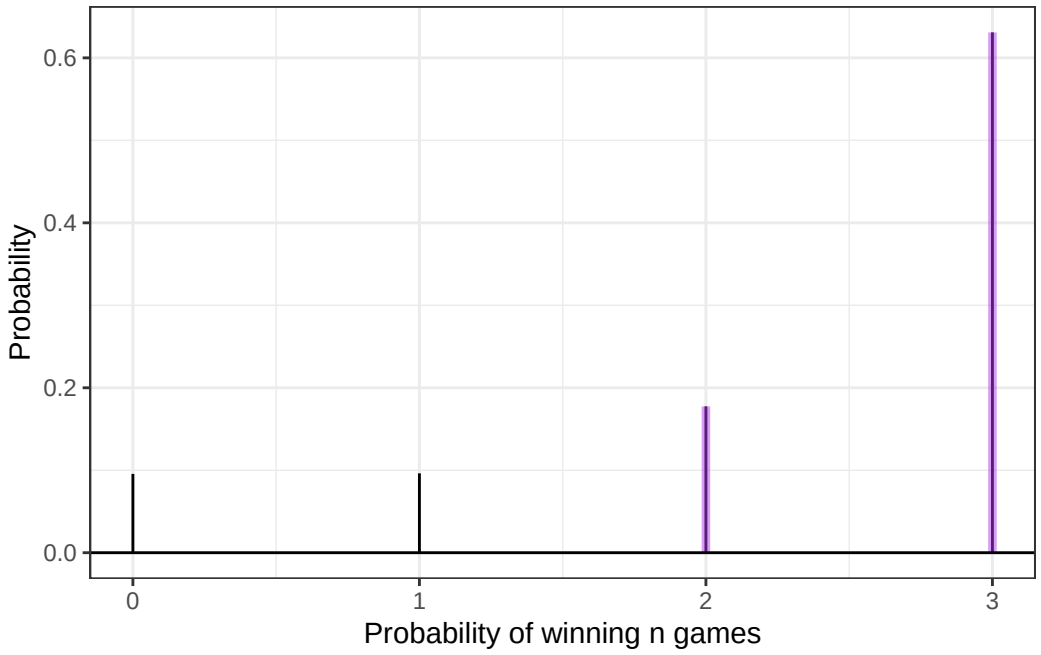
Hint: You can use the `integrate()` in `R` to compute the integral. Note that this requires the function to be vectorized – it should be unless you have added any block conditionals.

Solution

```r
# I don't know why this doesn't look like a code block in the Quarto editor, but it looks fine in the PDF.

daltbbinom = function(alpha, beta, theta, n, x){
  integrate(function(p){
    cvec = rowSums(sapply(0:n, function(i) choose(n,i) * p^i * (1-p)^(n-i) * theta^(i*(n-i))))
    altham   = (1/cvec) * choose(n,x) * p^x * (1-p)^(n-x) * theta^(x*(n-x))
```

```
    beta.pdf = dbeta(p, alpha, beta)
    altham * beta.pdf
  }, lower = 0, upper = 1)$value
}
```

```
# theta = 0.5
ggdat <- tibble(x = c(0,1,2,3)) |>
  mutate(PMF = sapply(x, function(k)
    daltbbinom(alpha=7, beta=3, theta=0.5, n=3, x=k)))

PMF.plot <- ggplot(ggdat) +
  geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x==2|x==3),
    aes(x = x, ymin = 0, ymax = PMF),
    color="darkorchid2", linewidth=1.5, alpha=0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("Probability of winning n games",
    breaks = seq(0, 3, 1)) +
  ylab("Probability") +
  theme_bw()

PMF.plot
```

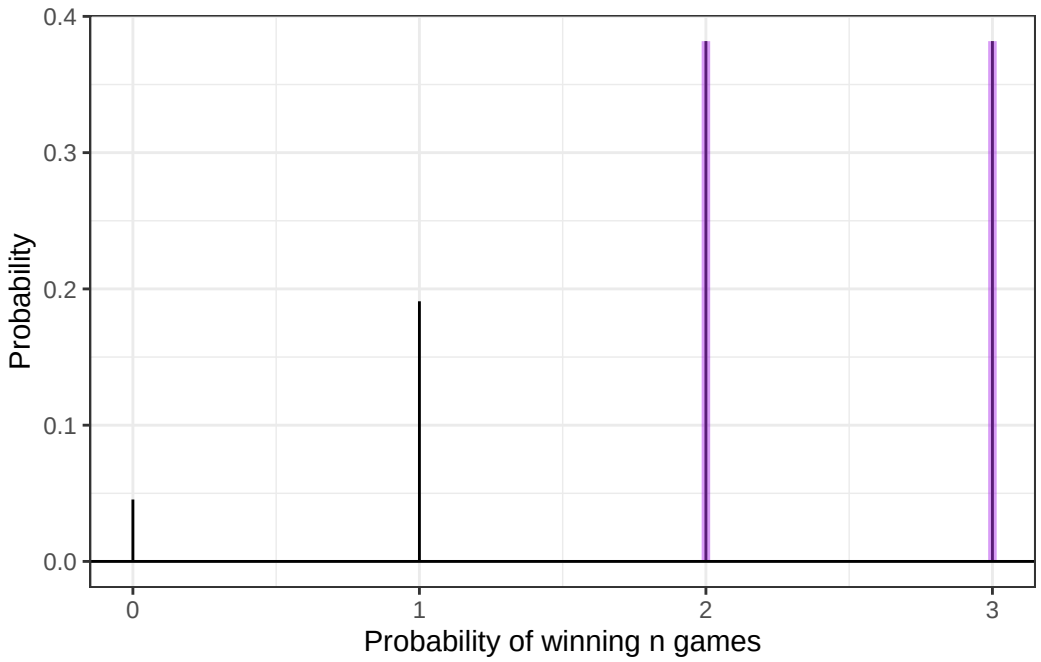



```
(A_win = 1 - daltbbinom(alpha=7, beta=3, theta=0.5, n=3, x=0) -
             daltbbinom(alpha=7, beta=3, theta=0.5, n=3, x=1))
```

```
[1] 0.8081866
```

```
# theta = 1
ggdat <- tibble(x = c(0,1,2,3)) |>
  mutate(PMF = sapply(x, function(k)
    daltbbinom(alpha=7, beta=3, theta=1, n=3, x=k)))

PMF.plot <- ggplot(ggdat) +
  geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x==2|x==3),
```

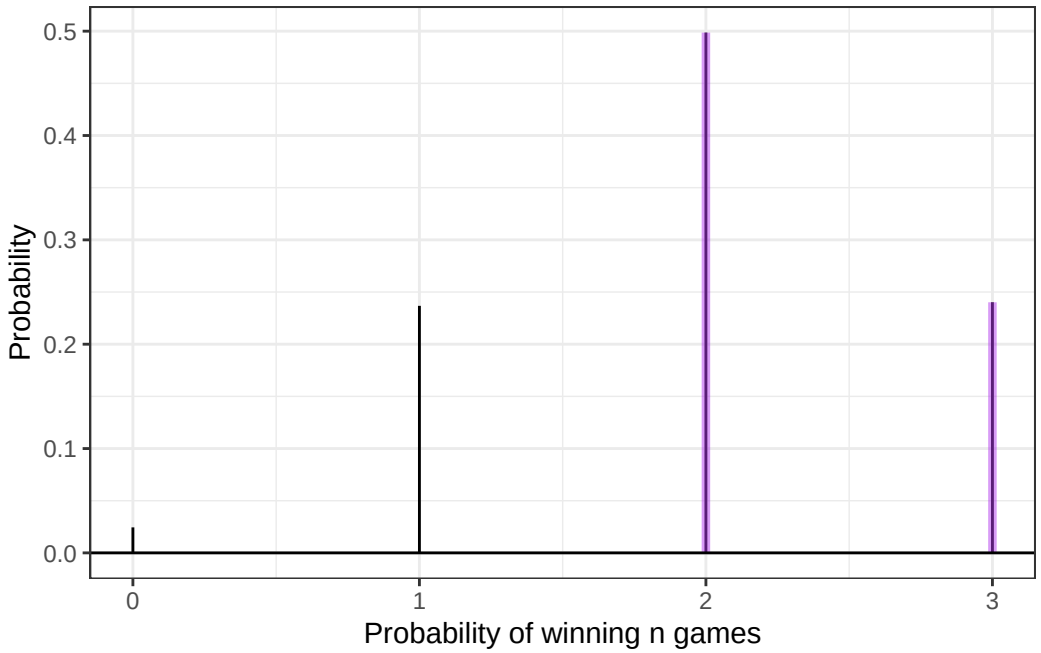
```
    aes(x = x, ymin = 0, ymax = PMF),
    color="darkorchid2", linewidth=1.5, alpha=0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("Probability of winning n games",
    breaks = seq(0, 3, 1)) +
  ylab("Probability") +
  theme_bw()

PMF.plot
```




```
(A_win = 1 - daltbbinom(alpha=7, beta=3, theta=1, n=3, x=0) -
             daltbbinom(alpha=7, beta=3, theta=1, n=3, x=1))
```

```
[1] 0.7636364
```

```
# theta = 1.5
ggdat <- tibble(x = c(0,1,2,3)) |>
  mutate(PMF = sapply(x, function(k)
    daltbbinom(alpha=7, beta=3, theta=1.5, n=3, x=k)))

PMF.plot <- ggplot(ggdat) +
  geom_linerange(aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange(data = ggdat |> filter(x==2|x==3),
    aes(x = x, ymin = 0, ymax = PMF),
    color="darkorchid2", linewidth=1.5, alpha=0.5) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("Probability of winning n games",
    breaks = seq(0, 3, 1)) +
  ylab("Probability") +
  theme_bw()

PMF.plot
```




```
(A_win = 1 - daltbbinom(alpha=7, beta=3, theta=1.5, n=3, x=0) -
             daltbbinom(alpha=7, beta=3, theta=1.5, n=3, x=1))
```

```
[1] 0.7387904
```

We now get a tighter spread for the variance with theta. Our probabilities are $\approx 0.81, 0.76$, and 0.74 for $\theta = 0.5, 1$, and 1.5, respectively. This clusters closesly around our results for the other distributions.

5 A Comparison

Compute the probabilities for all outcomes for Player A.

- Loses 0-2 (LL"L", LL"W")
- Loses 1-2 (WLL, LWL)
- Wins 2-0 (WW"W", WW"L")
- Wins 2-1 (LWW, WLW)

Note: You'll need to define these outcomes in terms of the random variable.

Create a bar plot (or column plot) using `ggplot` to show the probability of each outcome under the following conditions:

- **Binomial**
- **Negative Binomial**
  Note: You can compute the lose probabilities the same way as the win probabilities using success probability $(1 - p)$
- **Beta-Binomial**
- **Althman Binomial with $\theta = 0.50$ and $\theta = 1.5$**
- **Althman Beta-Binomial with $\theta = 0.50$ and $\theta = 1.5$**

Comment on the differences in the context of a best two-out-of-three match.

Solution

This seems easy to write a function for, but this is one of those examples where the long way is the short way. The need to distinguish between the third and fourth cases make this complicated. This approach also is very easy to analyze what is going on in terms of the probability statements. We check each distribution to make sure the sum of all outcomes is 1.

```r
# Binomial:
prob_binom = c(0)

prob_binom[1] = dbinom(x=0, prob=0.7, size=2)            # Lose 0-2
prob_binom[2] = dbinom(x=1, prob=0.7, size=2) * (1-0.7) # Lose 1-2
prob_binom[3] = dbinom(x=2, prob=0.7, size=2)            # Win 2-0
prob_binom[4] = dbinom(x=1, prob=0.7, size=2) * 0.7      # Win 2-1

sum(prob_binom)
```

```
[1] 1
```

```r
# Negative Binomial:
prob_nbinom = c(0)

prob_nbinom[1] = dbinom(x=0, prob=0.7, size=2)            # Lose 0-2
prob_nbinom[2] = dbinom(x=1, prob=0.7, size=2) * (1-0.7) # Lose 1-2
prob_nbinom[3] = dbinom(x=2, prob=0.7, size=2)            # Win 2-0
prob_nbinom[4] = dbinom(x=1, prob=0.7, size=2) * 0.7      # Win 2-1

sum(prob_nbinom)
```

```
[1] 1
```

```r
# Beta-Binomial:
prob_bbinom = c(0)

prob_bbinom[1] = dbbinom(x=0, size=2, alpha=7, beta=3)             # Lose 0-2
prob_bbinom[2] = dbbinom(x=1, size=2, alpha=7, beta=3) * (3/(7+3)) # Lose 1-2
prob_bbinom[3] = dbbinom(x=2, size=2, alpha=7, beta=3)             # Win 2-0
prob_bbinom[4] = dbbinom(x=1, size=2, alpha=7, beta=3) * (7/(7+3)) # Win 2-1

# NOTE: the $3/(7+3)$ is derived from the expected win probability of the Beta-Binomial distribution (alpha

sum(prob_bbinom)
```

```
[1] 1
```

```r
# Altham Binomial, theta = 0.5:
prob_altham05 = c(0)

prob_altham05[1] = daltbinom(p=0.7, theta=0.5, n=2, x=0)            # Lose 0-2
prob_altham05[2] = daltbinom(p=0.7, theta=0.5, n=2, x=1) * (1-0.7) # Lose 1-2
prob_altham05[3] = daltbinom(p=0.7, theta=0.5, n=2, x=2)            # Win 2-0
prob_altham05[4] = daltbinom(p=0.7, theta=0.5, n=2, x=1) * 0.7     # Win 2-1

sum(prob_altham05)
```

```
[1] 1
```

```r
# Altham Binomial, theta = 1.5:
prob_altham15 = c(0)

prob_altham15[1] = daltbinom(p=0.7, theta=1.5, n=2, x=0)            # Lose 0-2
prob_altham15[2] = daltbinom(p=0.7, theta=1.5, n=2, x=1) * (1-0.7) # Lose 1-2
prob_altham15[3] = daltbinom(p=0.7, theta=1.5, n=2, x=2)            # Win 2-0
prob_altham15[4] = daltbinom(p=0.7, theta=1.5, n=2, x=1) * 0.7     # Win 2-1

sum(prob_altham15)
```

```
[1] 1
```

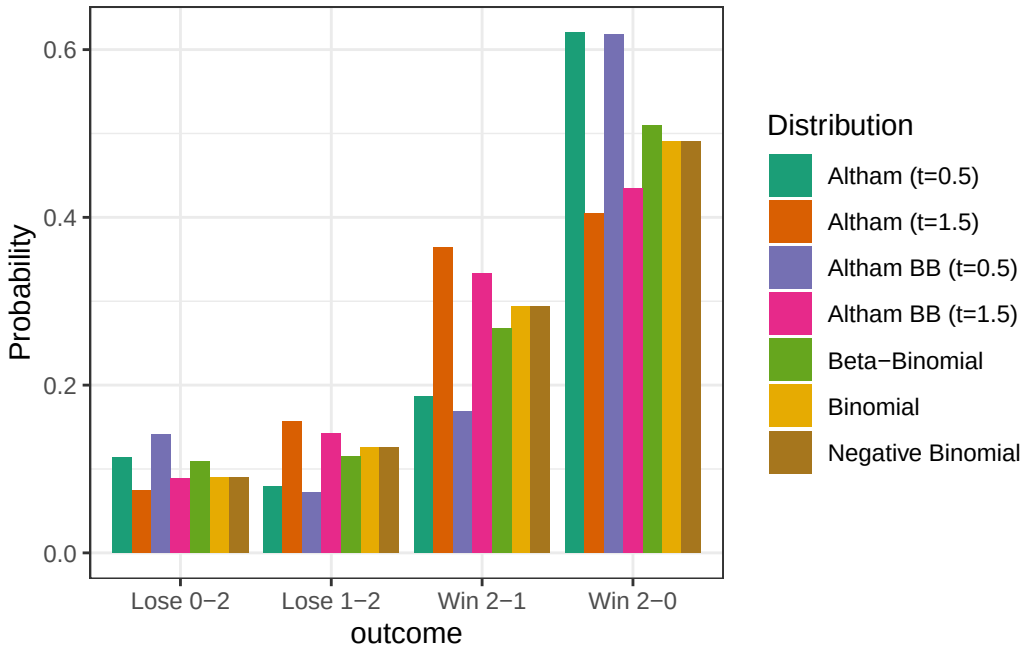
```r
# Altham Beta-Binomial, theta = 0.5:
prob_althambb05 = c(0)

prob_althambb05[1] = daltbbinom(alpha=7, beta=3, theta=0.5, n=2, x=0)              # Lose 0-2
prob_althambb05[2] = daltbbinom(alpha=7, beta=3, theta=0.5, n=2, x=1) * (3/(7+3)) # Lose 1-2
prob_althambb05[3] = daltbbinom(alpha=7, beta=3, theta=0.5, n=2, x=2)              # Win 2-0
prob_althambb05[4] = daltbbinom(alpha=7, beta=3, theta=0.5, n=2, x=1) * (7/(7+3)) # Win 2-1

sum(prob_althambb05)
```

```
[1] 1
```

```r
# Altham Beta-Binomial, theta = 1.5:
prob_althambb15 = c(0)

prob_althambb15[1] = daltbbinom(alpha=7, beta=3, theta=1.5, n=2, x=0)              # Lose 0-2
prob_althambb15[2] = daltbbinom(alpha=7, beta=3, theta=1.5, n=2, x=1) * (3/(7+3)) # Lose 1-2
prob_althambb15[3] = daltbbinom(alpha=7, beta=3, theta=1.5, n=2, x=2)              # Win 2-0
prob_althambb15[4] = daltbbinom(alpha=7, beta=3, theta=1.5, n=2, x=1) * (7/(7+3)) # Win 2-1

sum(prob_althambb15)
```

```
[1] 1
```

```r
plotting.dat = tibble(
  outcome = rep(c("Lose 0-2", "Lose 1-2", "Win 2-0", "Win 2-1"), 7),
  prob    = c(prob_binom, prob_nbinom, prob_bbinom,
              prob_altham05, prob_altham15,
              prob_althambb05, prob_althambb15),
  dist    = rep(c("Binomial", "Negative Binomial", "Beta-Binomial",
                  "Altham (t=0.5)", "Altham (t=1.5)",
                  "Altham BB (t=0.5)", "Altham BB (t=1.5)"), each=4)
)

ggplot(plotting.dat) +
  geom_col(aes(x = outcome, y = prob, fill = dist),
           position = "dodge") +
  scale_x_discrete(limits = c("Lose 0-2", "Lose 1-2", "Win 2-1", "Win 2-0")) +
  ylab("Probability") +
  scale_fill_brewer(palette = "Dark2", name = "Distribution") +
  theme_bw()
```




```
# This plot was informed by a lot of internet searches and minimal AI help
# (just the scale_x_discrete and the scale_fill_brewer).
# I really like the way it looks
# and I will probably come back to this for plotting anything similar.
```

The Binomial and Negative Binomial distributions are identical. The Beta-Binomial distributions have a higher probability of more "extreme" values, as their variance is higher (and, notably, they are also more left-skewed). Altham's for low theta, and then for larger theta, have the most extreme distributions. Low theta expects very extreme values, and high theta expects very moderate values. The same follows for Altham's Beta-Binomial distributions.

References

Wolodzko, Tymoteusz. 2026. *extraDistr: Additional Univariate and Multivariate Distributions*. https://doi.org/10.32614/CRAN.package.extraDistr.